

第三章 信任.

去中心化.

{ 匿名.
假名.

andré Blyper ?

1 BTC

0.01 cBTC , bit cent 分比特币

0.001 mBTC , millibitcoin 毫比特币.

0.00001 µBTC , bits 微比特币.

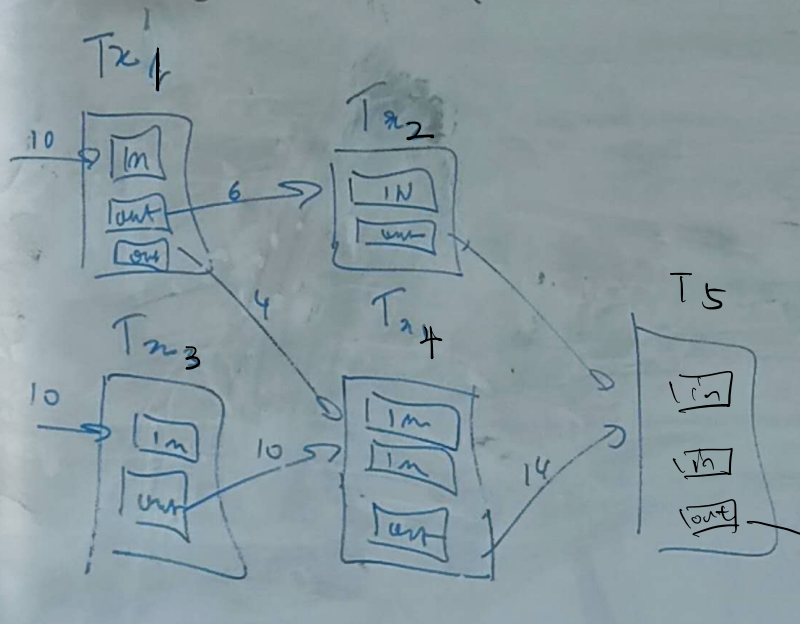
0.00000001 SAT

聪.

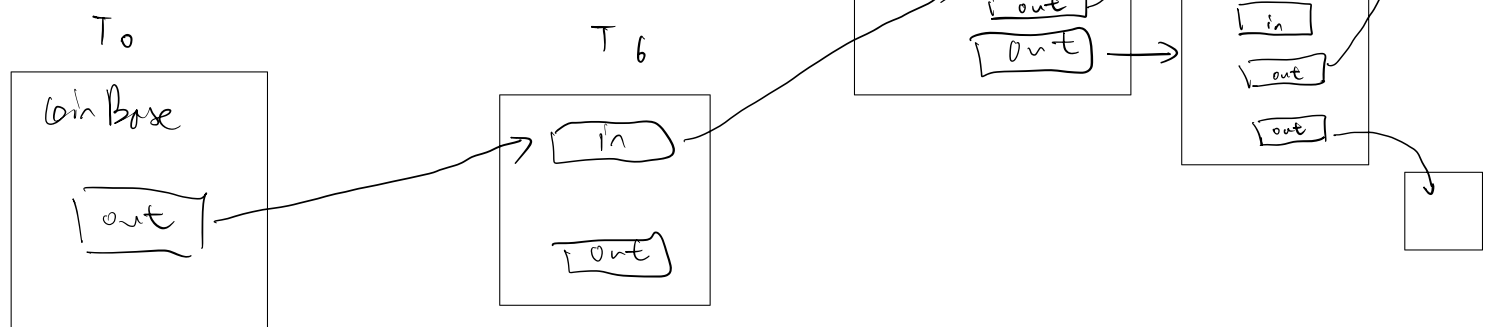
(最小单位)

2010.

交易类型.
Tx : type { - pays 支付 A → B
- Coinbase 挖矿 → M.



14:qt



SHA256

1. T_0 :

```
r
T0 : Sign_miner (Coinbase)
```

- 含义: T_0 表示一个 **Coinbase 交易**, 由矿工签名 (Sign_miner)。这是矿工挖出新区块后生成的新比特币奖励, 没有输入来源。

2. T_6 :

```
r
T6 : Sign_miner (pays (PubKey_alice), H(T0))
```

- 含义: T_6 是由矿工创建的一笔支付交易, 包含:
 - pays (PubKey_alice): 将比特币支付给 Alice 的公钥地址。
 - T_0 : 引用 Coinbase 交易 T_0 作为输入。

3. T_7 :

```
r
T7 : Sign_alice (pays (PubKey_bob), H(T6), H(T5))
```

- 含义: T_7 是 Alice 创建的一笔支付交易, 包含:
 - pays (PubKey_bob): 将比特币支付给 Bob 的公钥地址。
 - T_6 和 T_5 : 引用之前的两笔交易 T_6 和 T_5 作为输入。

3. 工作量证明 (PoW) 的过程

右下部分解释了 PoW 的流程:

1. 哈希计算:

- 用公式表示:

$$T_q(\text{Sha256}(B + \text{nonce})) \geq 2^k$$
 - B 是区块头的内容。
 - nonce 是随机数, 矿工会不断尝试不同的值。
 - 2^k 是挖矿的难度目标。
 - Sha256 是哈希函数, 用于生成固定长度的哈希值。

2. 过程描述:

- 初始值: nonce = 0。
- 矿工不断修改 nonce, 计算新的哈希值。
- 只有当 $\text{Sha256}(B + \text{nonce})$ 的结果满足目标难度 (即哈希值的前若干位是0), 才算找到有效区块。
- 图中标注的 "00000...00" 代表要求哈希值以多个0开头。

3. 迭代更新:

- 如果条件不满足, 增加 nonce, 重新计算哈希值, 直到找到符合要求的值。

4. T_8 :

```

r
复制代码
T8 : Sign_bob (pays (<Pub_charles, 0.1>, <Pub_bob, 0.4>), <T7>)

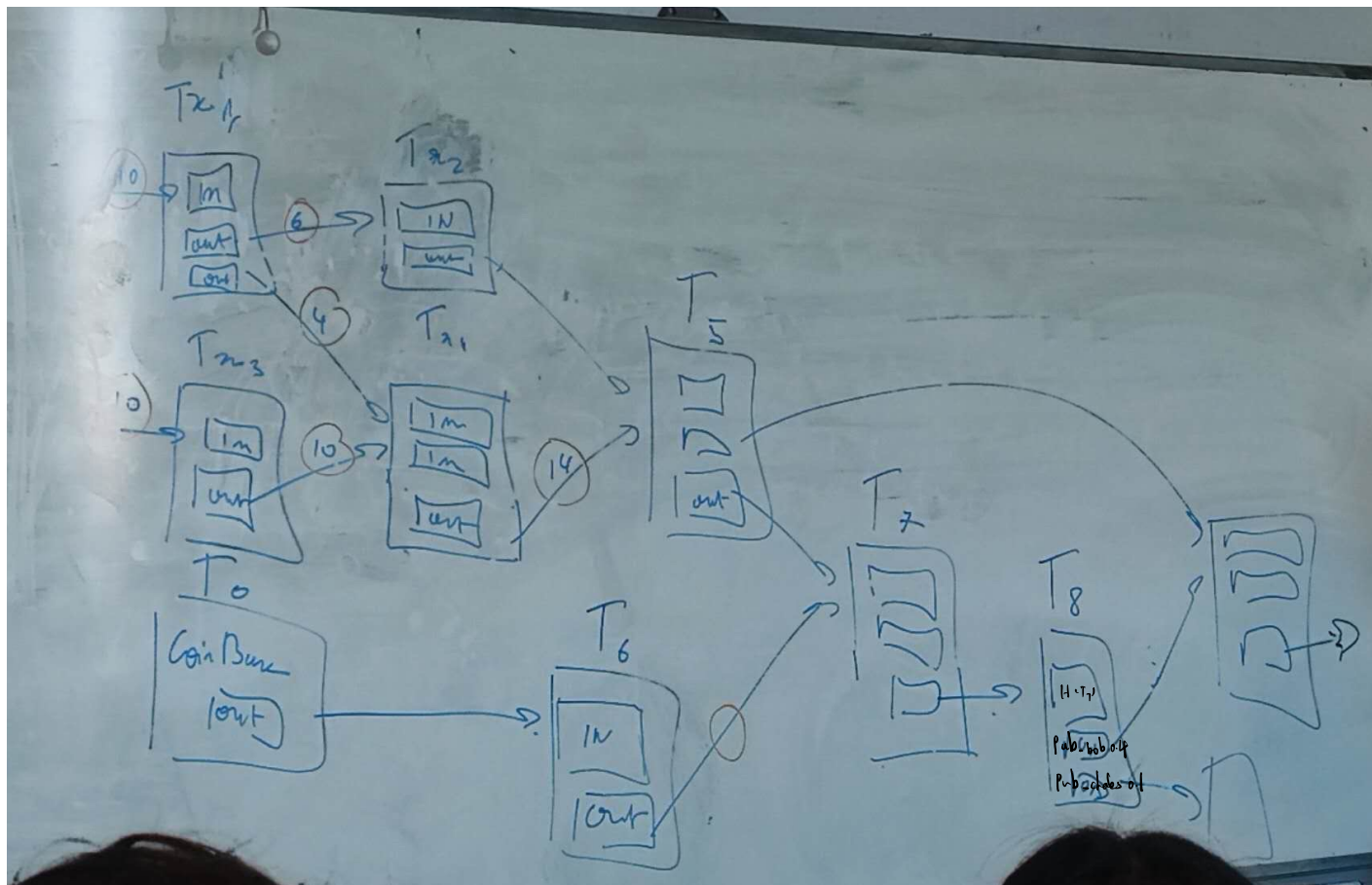
```

• 含义: T_8 是 Bob 发起的交易:

• pays: 支付了两个部分:

- $\langle \text{Pub_charles}, 0.1 \rangle$: 支付 0.1 的比特币给 Charles。
- $\langle \text{Pub_bob}, 0.4 \rangle$: 支付 0.4 的比特币回到 Bob 自己的地址。

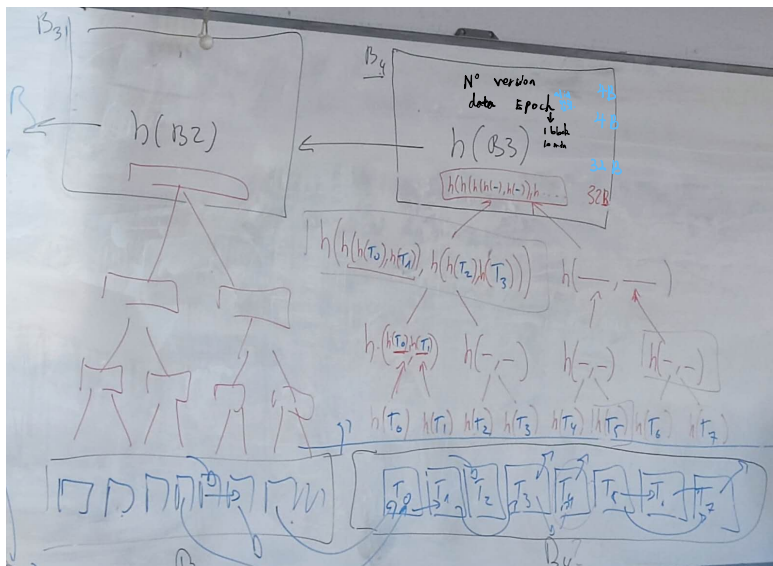
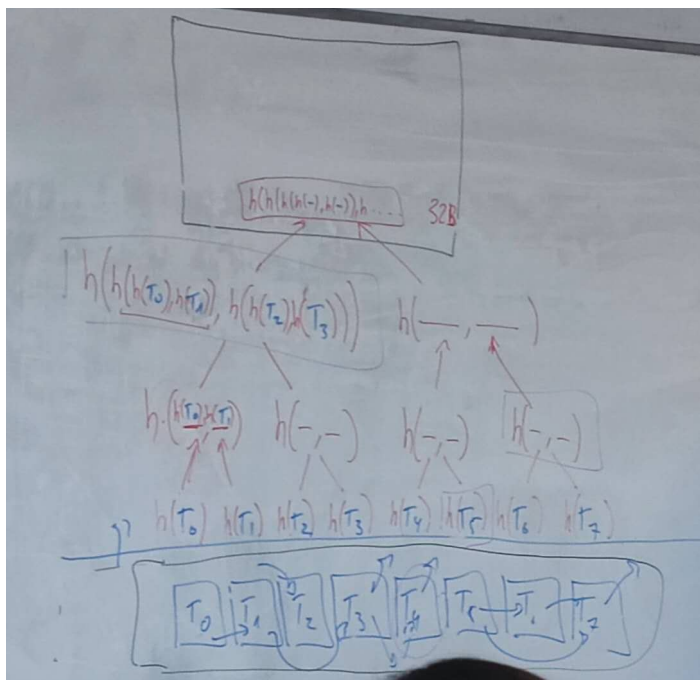
• T_7 : 引用了 Alice 的交易 T_7 作为输入。



默克尔树

Merkle Tree

二叉哈希树



每个区块哈希值

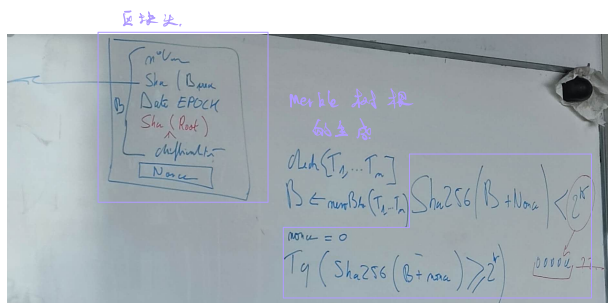
上一个区块的哈希

区块头是每个区块的关键部分，它包含描述区块信息的元数据，主要包括以下字段：

字段	大小	说明
Previous Block Hash	32 字节 (32b)	上一个区块的哈希值，用于链接到前一区块，确保链式结构的完整性。
Version	4 字节 (4b)	区块版本号，指示当前区块使用的规则或协议版本。
Timestamp	4 字节 (4b)	区块生成的时间戳，以 Unix 时间格式记录（秒数）。
Difficulty Target	4 字节 (4b)	当前区块的挖矿难度目标，定义了哈希值需要满足的条件（例如前导零个数）。
Merkle Root 梅克尔树根	32 字节 (32b)	当前区块交易数据的默克尔树根哈希值，表示区块中所有交易的摘要。
Nonce	4 字节 (4b)	用于 PoW（工作量证明）的随机数，矿工通过调整此值找到满足难度要求的哈希。

80
字节

SYDIT 攻击



1. 区块头 (Block Header) 的组成

左侧部分是一个典型的区块头的结构示意图，包含以下字段：

- 版本号 (m^v)：表明当前区块所使用的协议版本。
- 前区块的哈希值 ($\text{Sha}(\text{Prev})$)：当前区块引用的前一个区块的哈希值，用于将区块链连接起来。
- 时间戳 (Date / Epoch)：记录当前区块被创建的时间。
- Merkle Root ($\text{Sha}(\text{Root})$)：所有交易数据的Merkle树根哈希，用于验证区块中的交易完整性。
- 难度目标 (difficulty)：挖矿的难度目标，决定了找到有效哈希值需要的计算量。
- 随机数 (Nonce)：工作量证明中不断尝试的值，挖矿的核心变量。

2. Merkle树根的生成

右上角公式解释了Merkle树根的计算：

- 交易集合 ($T_1, T_2, \dots, T_n$)：区块中的所有交易。
- Merkle树 (MerkleTree)：通过将每笔交易的哈希值两两组合，递归地计算哈希值，最终得到一个根哈希值。

公式：

$$B \leftarrow \text{merkle_sha}(T_1, T_2, \dots, T_n)$$

B 是最后得到的 Merkle根。

3. 工作量证明 (PoW) 的过程

右下部分解释了 PoW 的流程：

1. 哈希计算：

- 用公式表示：

$$Tq(\text{Sha256}(B + \text{nonce})) \geq 2^k$$

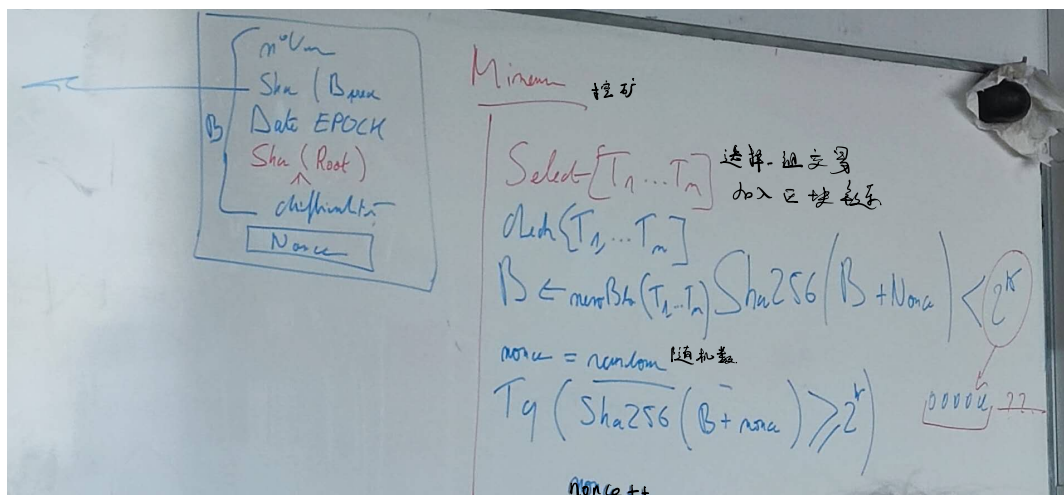
- B 是区块头的内容。
- nonce 是随机数，矿工会不断尝试不同的值。
- 2^k 是挖矿的难度目标。
- Sha256 是哈希函数，用于生成固定长度的哈希值。

2. 过程描述：

- 初始值：nonce = 0。
- 矿工不断修改 nonce，计算新的哈希值。
- 只有当 $\text{Sha256}(B + \text{nonce})$ 的结果满足目标难度（即哈希值的前若干位是0），才算找到有效区块。
- 图中标注的 "00000...00" 代表要求哈希值以多个0开头。

3. 迭代更新：

- 如果条件不满足，增加 nonce，重新计算哈希值，直到找到符合要求的值。



每个区块 50 比特币

↓

25

2012 年

↓

12.5

2016 年

↓

最后 - 枚

2140 年

↳ 区块奖励

小到接近 0.

21 万个区块.

减半 - 次

7:50 左右.

22:13 左右

2

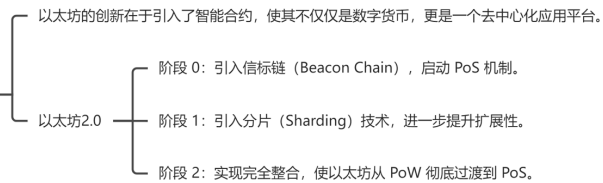


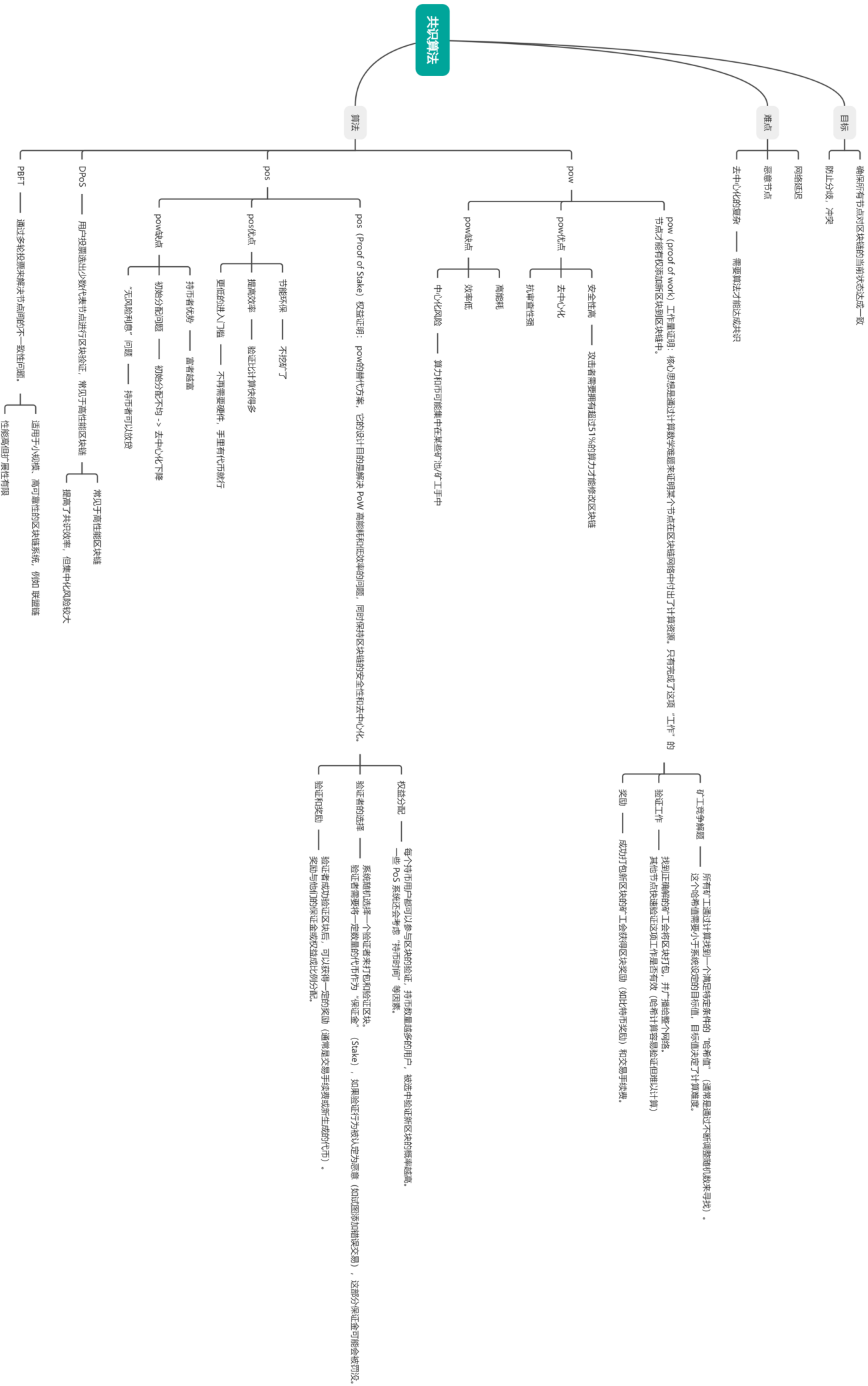
比特币/以太坊

比特币



以太坊





智能合约

